浙江大学
ZHEJIANG UNIVERSITY



计算机组成与设计

实验报告

实验名称浮点加操作指令及硬件扩展设计

实验地点 玉泉教 7-104

姓　名 郑智瑄

学　号 3230103913

实验日期 November 15, 2025

指导老师 赵武锋

Contents

1 实验概述 **1**

1.1 Project1 简介 1

1.2 实验环境 1

2 环境搭建 **1**

2.1 WSL2 安装与配置 1

2.1.1 安装步骤 1

2.2 VS Code 配置 2

2.3 工具链安装 2

2.3.1 Verilator 仿真器 2

2.3.2 GTKWave 波形查看器 2

2.3.3 Catapult-studio 编译平台（可选） 2

3 硬件设计与实现 **3**

3.1 译码级信号修改 3

3.1.1 el2_dec_decode_ctl.sv 修改 3

3.1.2 el2_dec.sv 修改 3

3.1.3 el2_exu.sv 修改 3

3.1.4 el2_veer.sv 修改 3

3.2 浮点加法器集成 4

3.2.1 el2_exu_div_ctl.sv 修改 4

4 仿真验证 **8**

4.1 测试文件 8

4.2 仿真步骤 9

4.2.1 生成仿真文件 9

4.2.2 运行仿真 9

4.2.3 查看波形 9

4.2.4 简化仿真流程（可选） 9

4.3 验证结果 10

4.3.1 完整波形分析 10

4.3.2 浮点加法细节波形 10
4.3.3 整数除法细节波形 11
4.3.4 信号复用机制验证 12
4.3.5 验证结论 12

5 实验探究 **14**
5.1 OR 运算数据保持方案的缺陷 14
5.1.1 问题提出 14
5.1.2 验证策略 15
5.1.3 波形分析 15
5.1.4 结论 17
5.2 优化方案实现 18
5.2.1 初步锁存方案 18
5.2.2 时序问题的根本原因 18
5.2.3 最终优化方案 19
5.2.4 状态机握手实现 22
5.3 乘除法器实现 26
5.3.1 测试代码设计 26
5.3.2 状态机集成过程 27
5.3.3 测试波形分析 30

6 遇到的问题与解决 **32**
6.1 Verilator 编译时序错误 32
6.1.1 问题描述 32
6.1.2 问题分析 32
6.1.3 解决方案 32
6.1.4 解决结果 33
6.2 浮点寄存器初始化错误 34
6.2.1 问题发现 34
6.2.2 问题分析 35
6.2.3 解决方案 36

7 实验总结与实验心得 **37**
7.1 实验完成情况 37

7.2 实验总结 38
7.3 实验心得 38

1、实验概述

1.1 Project1 简介

本次 project 基于著名的开源 SoC（System on Chips）项目 VeeRwolf，VeeRwolf 所采用的 RISC-V 核心是 EL2，也是著名的开源项目。

EL2 是一个四级流水线的简单但完整的 RISC-V 处理器，支持 RV32IMC 指令集，但是不原生支持浮点操作。本 project 需要按照指示，完成 EL2 的指令集及硬件集成，使之支持单精度的基于 IEEE 754 标准的浮点加操作。

1.2 实验环境

- 操作系统：Windows 11 + WSL2 (Ubuntu22.04)
- 开发工具：VS Code + WSL 扩展
- 仿真器：Verilator
- 波形查看器：GTKWave
- 编译平台：Catapult-studio (可选)
- 硬件描述语言：SystemVerilog

2、环境搭建

2.1 WSL2 安装与配置

WSL2 (Windows Subsystem for Linux 2) 是 Windows 10/11 支持的轻量级 Linux 虚拟机，非常适合中小型体量的 Linux 工程搭建和运行。

2.1.1 安装步骤

1. 以管理员身份打开 PowerShell
2. 执行命令 `wsl --install`，启用 WSL 及虚拟机功能，下载 WSL2 内核并安装 Ubuntu 发行版
3. 根据提示设置 Ubuntu 用户名和密码
4. 执行 `wsl --list` 验证安装成功
5. 执行 `wsl` 或 `bash` 进入 Ubuntu 系统
6. 执行 `sudo apt update` 更新软件包列表
7. 执行 `sudo apt install -y build-essential` 安装基本开发工具

2.2 VS Code 配置

1. 在 VS Code 中安装 WSL 扩展和 Remote Explorer 扩展
2. 通过 Remote Explorer 连接到 Ubuntu 虚拟机
3. 将工程文件夹挂载到 WSL 环境中

挂载路径转换规则：将 Windows 路径中的 \ 替换为 /，将盘符 C: 替换为 /mnt/c。
例如:Windows 路径：`C:\Users\xxx\RVfpga_NexysA7-DDR`
转换为 WSL 路径：`/mnt/c/Users/xxx/RVfpga_NexysA7-DDR`

2.3 工具链安装

2.3.1 Verilator 仿真器

```
sudo apt-get install git make autoconf g++ flex bison libfl2 libfl-dev
sudo apt-get install verilator
verilator --version # 验证安装
```

2.3.2 GTKWave 波形查看器

```
sudo apt-get install gtkwave
gtkwave # 验证安装
```

2.3.3 Catapult-studio 编译平台（可选）

```
mv /mnt/c/path/to/catapult-sdk_1.9.1.deb /home/username/
cd ~
sudo apt install ./catapult-sdk*.deb
cd ~/catapult-sdk_examples/1.9.1
source ./set_sdk_path.sh
catapult-studio # 验证安装
```

3、硬件设计与实现

3.1 译码级信号修改

为了支持浮点加指令，需要在译码级添加相应的控制信号 `fp_p`。

3.1.1 el2_dec_decode_ctl.sv 修改

在输出信号中添加 `fp_p` 信号：

```
output logic fp_p,
```

3.1.2 el2_dec.sv 修改

在输出信号中添加 `fp_p` 信号：

```
output logic fp_p,
```

3.1.3 el2_exu.sv 修改

在输入信号中添加 `fp_p` 信号，并在除法器例化时连接该信号：

```
input logic fp_p,

el2_exu_div_ctl divider(
    ...
    .fp(fp_p),
    ...
);
```

3.1.4 el2_veer.sv 修改

在连接信号中添加 `fp_p` 信号，确保信号从顶层模块正确传递。

3.2 浮点加法器集成

3.2.1 el2_exu_div_ctl.sv 修改

将浮点加法器模块 fp_adder 集成到整数除法器模块 el2_exu_div_ctl 中，复用除法器的控制信号和数据通路。使用提供的修改模块 el2_exu_div_ctl_2.sv。

关键设计要点：

1. 操作数锁存：由于 dividend 和 divisor 信号只持续 1-2 个时钟周期，而浮点加法器需要多个周期读取操作数，因此需要添加寄存器链进行锁存
2. 握手信号生成：通过寄存器链延拓 stb 信号，生成 stb_for_fp，确保浮点加法器能够正确捕获操作数
3. 操作类型识别：设计 fp_operation_latch 模块，根据 dp.valid、fp.fp_add 等信号识别当前操作类型
4. 完成信号复用：使用 active_operation 信号选择浮点加法器的 fpadd_done 或整数除法器的 finish_dly
5. 结果输出复用：根据操作类型，选择输出浮点加法结果 out_fpadd 或整数除法结果 out_raw

主要修改内容：

1. 信号定义　首先定义浮点加法器所需的信号和寄存器链：

```
wire [31:0] out_fpadd;       // 浮点加法输出结果
wire fpadd_done;             // 浮点加法完成信号
wire stb;                    // 握手信号（strobing signal）
reg stb_d, stb_dd;           // 握手信号的1级和2级延时
wire stb_for_fp;             // 延拓后的握手信号
reg [31:0] divisor_d, divisor_dd; // divisor的1级和2级寄存器
reg [31:0] dividend_d;          // dividend的1级寄存器
wire [31:0] dividend_for_fp; // 延拓后的操作数A
wire [31:0] divisor_for_fp; // 延拓后的操作数B
```

说明：这些信号用于解决操作数持续时间短的问题。原始的 dividend 和 divisor 信号只在 1-2 个时钟周期内有效，而浮点加法器的状态机需要多个周期来读取操作数。

2. 操作数锁存　使用寄存器链锁存操作数和握手信号：

```
// 生成握手信号
```

```
assign stb = fp.fp_add;

// 在时钟上升沿锁存信号
always @(posedge clk) begin
    stb_d <= stb;           // 握手信号延时1个时钟周期
    stb_dd <= stb_d;        // 握手信号延时2个时钟周期
    divisor_d <= divisor;   // divisor延时1个时钟周期
    divisor_dd <= divisor_d; // divisor延时2个时钟周期
    dividend_d <= dividend; // dividend延时1个时钟周期
end
```

说明：通过寄存器链保存历史数据，形成 3 个时钟周期的数据窗口。stb 信号在浮点加法操作有效时拉高。

3. 信号延拓　使用或运算延长信号的有效时间：

```
// 将3个时钟周期的信号通过或运算合并
assign stb_for_fp = stb | stb_d | stb_dd;
assign divisor_for_fp = divisor | divisor_d | divisor_dd;
assign dividend_for_fp = dividend | dividend_d;
```

说明：或运算确保只要历史窗口中任何一个周期有数据，延拓后的信号就保持有效。这样 stb_for_fp 可以维持 3 个时钟周期，给浮点加法器充足的时间读取操作数。

4. 浮点加法器例化　实例化 fp_adder 模块并连接信号：

```
fp_adder simple_fp_adder(
    .input_a(dividend),       // 操作数A：直接使用dividend
    .input_b(divisor_for_fp), // 操作数B：使用延拓后的divisor
    .input_a_stb(stb_for_fp), // A的握手信号：延拓后的stb
    .input_b_stb(stb_for_fp), // B的握手信号：延拓后的stb
    .output_z_ack(1),         // 输出应答：始终准备接收（tied to 1）
    .clk(clk),                // 时钟信号
    .rst(~rst_l),             // 复位信号：低电平有效转高电平有效
    .output_z(out_fpadd),     // 输出：浮点加法结果
    .output_z_stb(fpadd_done), // 输出有效信号：表示计算完成
    .input_a_ack(),           // A的应答信号：悬空（未使用）
    .input_b_ack()            // B的应答信号：悬空（未使用）
);
```

说明：fp_adder 使用握手协议，_stb 信号表示数据有效，_ack 信号表示准备好接收。output_z_ack 设为 1 表示始终准备接收结果。

5. 操作类型识别　实例化操作类型锁存模块：

```
wire [3:0] active_operation;
fp_operation_latch op_latch(
    .clk(clk),
    .dp_valid(dp.valid),      // 除法有效信号
    .fp_add(fp.fp_add),       // 浮点加法使能
    .fp_mul(fp.fp_mul),       // 浮点乘法使能（预留扩展）
    .fp_div(fp.fp_div),       // 浮点除法使能（预留扩展）
    .a(active_operation)      // 输出独热码：[3]整除 [2]浮加 [1]浮乘 [0]浮除
);
```

说明：fp_operation_latch 模块根据输入的使能信号，输出独热码表示当前活跃的操作类型。这个模块会锁存操作类型，直到操作完成。

6. 完成信号多路选择　根据操作类型选择相应的完成信号：

```
assign finish_dly_div_fp =
    ({32{active_operation[3]}} & {32{finish_dly}}) | // 整数除法完成
    ({32{active_operation[2]}} & {32{fpadd_done}}); // 浮点加法完成
```

说明：使用位复制（{32{bit}}）将单个控制位扩展为 32 位掩码，然后通过与运算和或运算实现多路选择。当 active_operation[3]=1 时选择 finish_dly，当 active_operation[2]=1 时选择 fpadd_done。

7. 结果输出多路选择　根据操作类型选择相应的输出结果：

```
assign out[31:0] =
    ({32{active_operation[3]}} & {32{finish_dly}} & out_raw[31:0]) | //
        整数除法结果
    ({32{active_operation[2]}} & {32{fpadd_done}} & out_fpadd[31:0]); //
         浮点加法结果
```

说明：采用双重掩码机制：第一层掩码根据操作类型选择通路，第二层掩码确保只有在完成信号有效时才输出结果。这种设计避免了输出中间状态的错误数据。

设计说明：

- 寄存器链设计：通过 3 级寄存器链（`stb`、`stb_d`、`stb_dd`）延长握手信号有效时间，确保浮点加法器的状态机能够稳定读取操作数
- 或逻辑延拓：使用或运算将多个时钟周期的操作数合并，保证操作数在浮点加法器读取期间始终有效
- 操作类型编码：`active_operation[3:0]` 采用独热码，`[3]` 表示整数除法，`[2]` 表示浮点加法，`[1][0]` 预留给浮点乘除
- 信号复用：通过位掩码（bit mask）实现完成信号和输出结果的多路选择，避免使用额外的 MUX

4、仿真验证

4.1 测试文件

测试文件 `fp_add.S` 是一个 RISC-V 汇编程序，用于测试浮点加法器和整数除法器的功能。主要内容如下：

```
#define INSERT_NOPS_1  nop;
#define INSERT_NOPS_2  nop; INSERT_NOPS_1
#define INSERT_NOPS_3  nop; INSERT_NOPS_2
#define INSERT_NOPS_4  nop; INSERT_NOPS_3
#define INSERT_NOPS_5  nop; INSERT_NOPS_4
#define INSERT_NOPS_6  nop; INSERT_NOPS_5
#define INSERT_NOPS_7  nop; INSERT_NOPS_6
#define INSERT_NOPS_8  nop; INSERT_NOPS_7
#define INSERT_NOPS_9  nop; INSERT_NOPS_8
#define INSERT_NOPS_10 nop; INSERT_NOPS_9

.globl main
main:
li t3, 0x40800000    # Load float 4.0 into t3
li t4, 0x40000000    # Load float 2.0 into t4
li a3, 20            # Load integer 20 into a3 (dividend)
li a4, 4             # Load integer 4 into a4 (divisor)
REPEAT:
INSERT_NOPS_4        # Insert 4 NOPs for pipeline spacing
.word 0x01ce8f53     # fadd.s f30, t4, t3 (2.0 + 4.0 = 6.0)
INSERT_NOPS_4        # Insert 4 NOPs for pipeline spacing
div a5, a3, a4       # Test integer division: a5 = 20 / 4 = 5
INSERT_NOPS_10       # Insert 10 NOPs for pipeline spacing
beq zero, zero, REPEAT # Infinite loop to repeat the test
```

代码说明：

- **NOP** 宏定义：用于在流水线中插入空指令，确保非流水线模块有足够时间完成操作
- 浮点数加载：`0x40800000` 表示 4.0，`0x40000000` 表示 2.0（IEEE 754 格式）
- **fadd.s** 指令：自定义的浮点加指令，编码为 `0x01CE8F53`，计算 $2.0 + 4.0 = 6.0$
- **div** 指令：标准整数除法指令，验证原除法器功能未受影响
- 循环执行：通过无条件跳转实现重复测试

4.2 仿真步骤

4.2.1 生成仿真文件

```
cd Simulators/verilatorSIM_Trace
make
./obj_dir/Vrvfpgasim
```

编译成功表示修改的代码没有语法错误。

4.2.2 运行仿真

```
cd RVfpga_NexysA7-DDR/Labs/Lab18/commandLine
../../../Simulators/verilatorSIM_Trace/Vrvfpgasim \
    +ram_init_file=fp_add.vh +vcd=1
# 等待3秒后按 Ctrl+C 终止
```

此步骤生成波形文件 `trace.vcd`。

4.2.3 查看波形

```
gtkwave trace.vcd
# 在GTKWave中通过 File -> Read TCL Script File
# 读取 test.tcl 显示预设信号
```

4.2.4 简化仿真流程（可选）

为了简化仿真操作，我在 `Labs/Lab18/commandLine/Makefile` 中添加了自定义的 Make 目标：

```
vcd:
    ../../../Simulators/verilatorSIM_Trace/Vrvfpgasim \
    +ram_init_file=fp_add.vh +vcd=1

gtk:
    gtkwave trace.vcd
```

添加后，可以使用更简洁的命令进行仿真：

```
# 进入测试目录
cd RVfpga_NexysA7-DDR/Labs/Lab18/commandLine

# 运行仿真并生成波形文件
make vcd
# 等待3秒后按 Ctrl+C 终止

# 打开GTKWave查看波形
make gtk
```

4.3 验证结果

通过 GTKWave 查看仿真生成的波形文件，可以直观地观察到浮点加法器和整数除法器的工作状态。本节通过三张波形图展示验证过程：图1展示完整的指令执行流程，图2和图3分别展示浮点加法和整数除法的细节波形。

4.3.1 完整波形分析

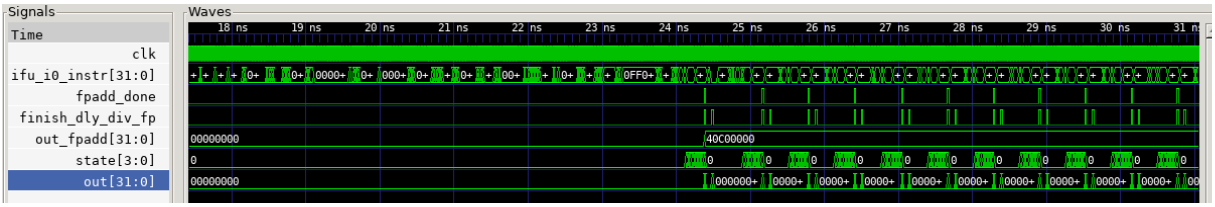


Figure 1: 完整指令执行流程波形图

图1展示了从 18ns 到 31 ns 时间段内的完整仿真波形，包含了浮点加法指令和整数除法指令的连续执行。

4.3.2 浮点加法细节波形

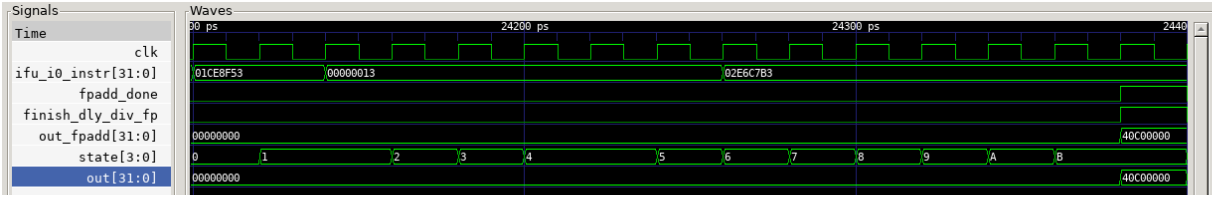


Figure 2: 浮点加法运算细节波形（24.1 ns - 24.4 ns）

图2放大显示了浮点加法运算完成阶段的细节波形。

测试参数：

- 指令编码：0x01CE8F53（fadd.s f30, t4, t3）
- 操作数 **A**：0x40800000（IEEE 754 格式，十进制 4.0）
- 操作数 **B**：0x40000000（IEEE 754 格式，十进制 2.0）
- 预期结果：0x40C00000（IEEE 754 格式，十进制 6.0）

波形细节分析：

- 状态转换：状态机从状态 6→7→8→9→A→B→0，完成最后阶段的规格化和舍入操作
- 输出结果：out_fpadd[31:0] = 0x40C00000
- 完成信号：fpadd_done 在状态机回到 0 时拉高，表示计算完成
- 输出稳定：out[31:0] 信号稳定输出 0x40C00000，随后在整数除法指令到来时变回 0x00000000

结果验证：

$$4.0 + 2.0 = 6.0$$

根据 IEEE 754 标准：

- 0x40800000 $= (-1)^0 \times 2^{129-127} \times 1.0 = 4.0$
- 0x40000000 $= (-1)^0 \times 2^{128-127} \times 1.0 = 2.0$
- 0x40C00000 $= (-1)^0 \times 2^{129-127} \times 1.5 = 6.0$

计算结果完全正确！

4.3.3 整数除法细节波形

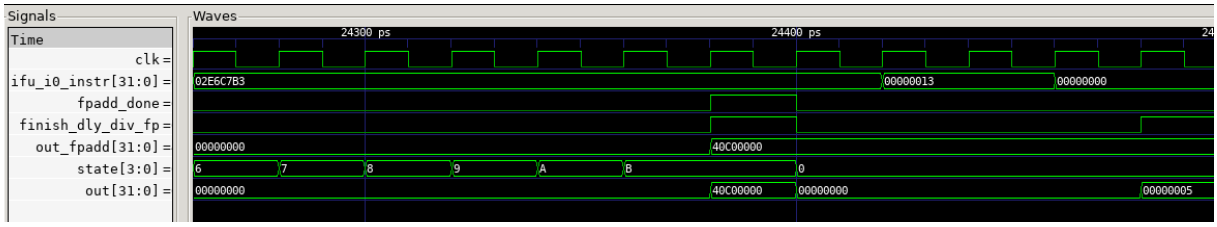


Figure 3: 整数除法运算细节波形（24.3 ns - 24.5 ns）

图3展示了整数除法指令执行期间的细节波形，包含了高频信号的完整状态变化。

测试参数：

- 指令编码：0x02E6C7B3（div a5, a3, a4）

- 被除数：0x00000014（十进制 20）
- 除数：0x00000004（十进制 4）
- 预期结果：0x00000005（十进制 5）

波形细节分析：

- 指令识别：在约 24 ns 时刻，ifu_i0_instr 显示 0x02E6C7B3（div 指令）
- 高频活动：波形显示了除法器内部的迭代计算过程，ifu_i0_instr 在后续周期出现高频变化
- 完成信号：finish_dly_div_fp 信号多次拉高，表示除法器完成多次迭代计算
- 输出结果：out[31:0] 在除法完成后显示 0x00000005（十进制 5）
- 状态保持：浮点加法器 state 保持在 0 状态，out_fpadd 显示 0x40C00000（保持上次结果）

结果验证：

$$20 \div 4 = 5$$

整数除法运算结果正确，且原除法器功能未受浮点加法器集成的影响。

4.3.4 信号复用机制验证

从三张波形图的对比分析可以看出信号复用机制的正确性：

- 浮点加法阶段（图2）：
  - out[31:0] 输出浮点加法结果 0x40C00000
  - finish_dly_div_fp 由 fpadd_done 信号控制
- 整数除法阶段（图3）：
  - out[31:0] 切换输出整数除法结果 0x00000005
  - finish_dly_div_fp 由除法器的 finish_dly 信号控制

4.3.5 验证结论

基于以上三个层次的波形分析，得出以下验证结论：

1. 浮点加法器功能正确：成功实现 IEEE 754 标准的单精度浮点加法运算，对阶、尾数相加、规格化、舍入等步骤均正确执行
2. 状态机运行稳定：浮点加法器状态机完整经历 12 个状态（0→1→...→B→0），状态转换逻辑正确，无死锁或错误跳转

3. 完成信号可靠：`fpadd_done` 和 `finish_dly_div_fp` 信号时序正确，保证了流水线的正确控制
4. 原有功能完整：整数除法器功能未受影响，迭代计算过程正常，结果准确
5. 信号复用成功：`out` 信号和 `finish_dly_div_fp` 信号的多路选择机制工作正常，能够根据操作类型正确切换输出
6. 性能符合预期：浮点加法耗时约 10 余个时钟周期，符合非流水线设计的性能特征

综合验证结果表明：浮点加法器已成功集成到 EL2 处理器中，硬件功能正确，信号时序准确，完全实现了预期的设计目标。

5、实验探究

5.1 OR 运算数据保持方案的缺陷

5.1.1 问题提出

在将浮点加法器集成到 el2_exu_div_ctl.sv 时，我们采用了与整数除法器共享输入端口的设计。为了解决浮点加法器握手协议延迟导致的数据对齐问题，最初采用了延迟链 + **OR** 运算的数据保持方案：

```
// 延迟链：保存历史数据
reg [31:0] divisor_d, divisor_dd;
reg [31:0] dividend_d;

always@(posedge clk) begin
    divisor_d <= divisor;
    divisor_dd <= divisor_d;
    dividend_d <= dividend;
end

// OR运算：尝试保持有效数据
assign divisor_for_fp = divisor | divisor_d | divisor_dd;
assign dividend_for_fp = dividend | dividend_d;
```

设计初衷：

- dividend 和 divisor 是组合逻辑信号，每个周期都可能变化
- fp_adder 使用握手协议，需要 2-3 个时钟周期才能锁存数据
- 使用延迟链保存多个周期的历史值，通过 OR 运算确保数据保持

理论上的问题：当新指令到达时，如果新旧数据都非零，OR 运算会导致数据污染：

```
// 例如：浮点数 4.0 后紧跟整数 4
0x40800000 (浮点数 4.0，历史数据)
|
0x00000004 (整数 4，新数据)
==========
0x40800004 (错误的混合结果！)
```

这种混合值既不是有效的浮点数，也不是正确的整数，可能导致计算结果错误。

5.1.2 验证策略

为了验证 OR 运算方案是否会导致数据污染，我们设计了零延迟背靠背测试：

测试程序 `fp_conflict.S`：

```
main:
    li t3, 0x40800000 # x28/f28 = 4.0
    li t4, 0x40000000 # x29/f29 = 2.0
    li a3, 20         # 被除数
    li a4, 4          # 除数

CRITICAL_TEST:
    INSERT_NOPS_4
    .word 0x01ce8f53 # fadd.s f30, f29, f28 (4.0+2.0=6.0)
    div a5, a3, a4    # 20/4=5 (紧随其后，零延迟)
    INSERT_NOPS_20
```

测试设计要点：

- 使用 x28-x30 初始化数据（利用与 f28-f30 共享物理寄存器的特性）
- `fadd.s` 和 `div` 连续执行，无任何延迟
- 浮点加法需要 10+ 周期，除法指令会在浮点运算期间到达
- 关键冲突点：
  - 浮点加法使用 `divisor = 0x40800000`（4.0）和 `divisor = 0x40000000`（2.0）
  - 整数除法使用 `divisor = 0x00000004`（4）
  - 如果 OR 运算存在问题，会产生 `0x40800000 | 0x00000004 = 0x40800004`

5.1.3 波形分析

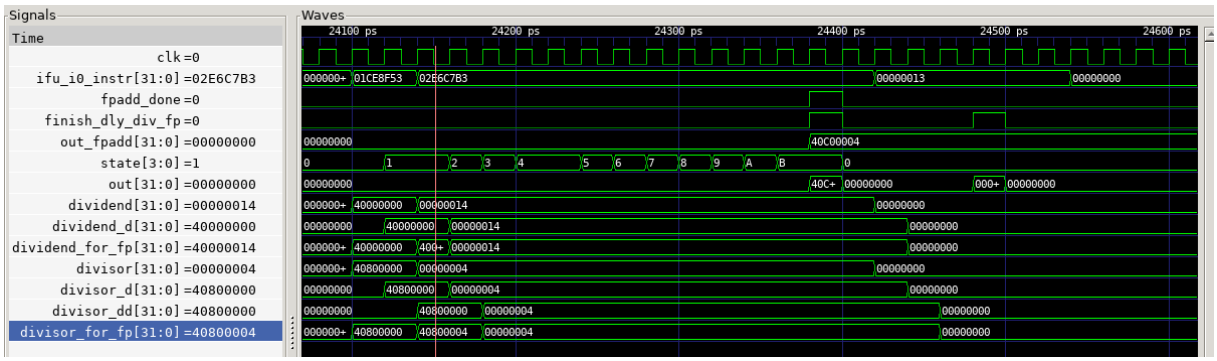


Figure 4: OR 运算数据污染现象 1

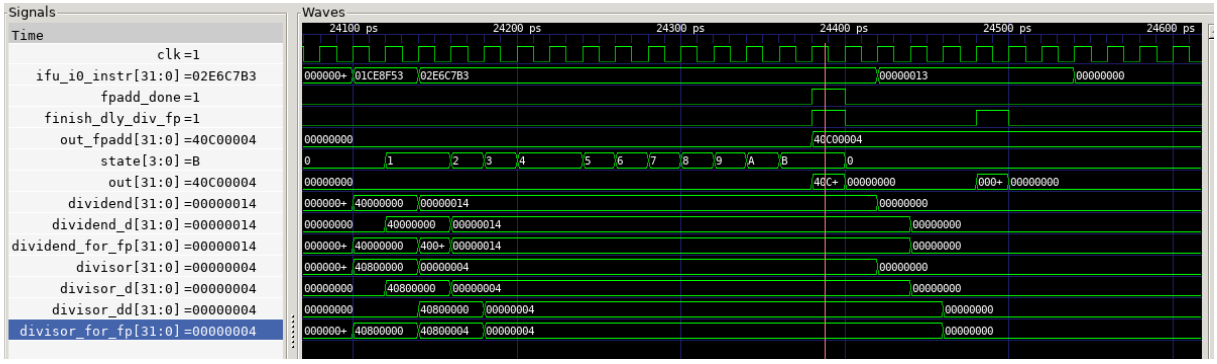


Figure 5: OR 运算数据污染现象 2

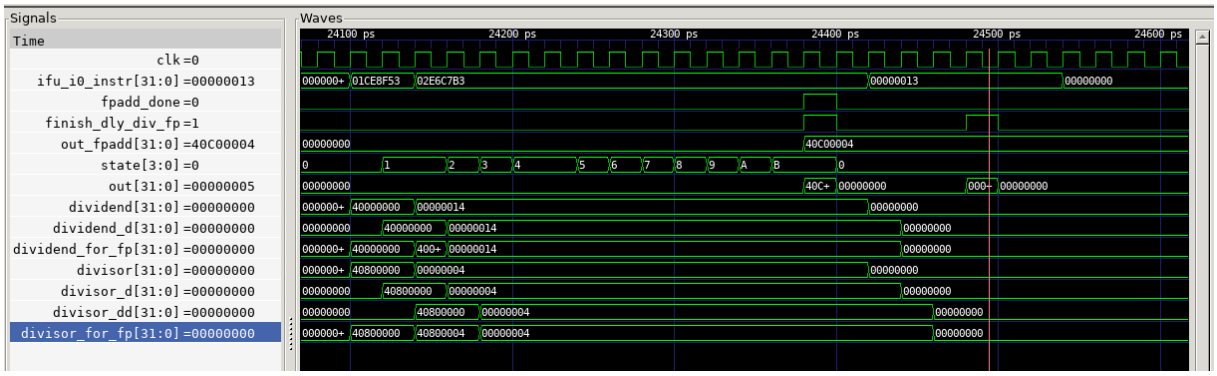


Figure 6: OR 运算数据污染现象 3

关键信号观察（图4、图5 和图6）：

1. 指令序列（图4）：
   - 24200ps 附近：`ifu_i0_instr = 0x01CE8F53`（fadd.s 指令）
   - 24300ps 附近：`ifu_i0_instr = 0x02E6C7B3`（div 指令）
   - 两条指令间隔极短，形成零延迟背靠背执行
2. 数据污染过程（图5）：
   - 初始阶段：`divisor_for_fp = 0x40800000`（浮点数 4.0，正确）
   - **div** 指令进入后：
     - `divisor` 变为 `0x00000004`（整数 4）
     - `divisor_d` 仍为 `0x40800000`（上一周期的浮点数）
     - `divisor_for_fp = 0x40800000 | 0x00000004 = 0x40800004` 数据污染！
   - 后续周期：`divisor_for_fp` 逐渐清零（`0x00000004 | 0x00000000`）
   - `dividend_for_fp` 也出现类似污染现象

3. 计算结果错误（图6）：

   - 预期结果：$4.0 + 2.0 = 6.0 =$ `0x40C00000`（IEEE 754）
   - 实际结果：`out_fpadd = 0x40C00004`
   - 误差来源：输入被污染为 `0x40800004`（ 4.0000047）
   - 计算过程：$4.0000047 + 2.0 = 6.0000047 \approx$ `0x40C00004` 逻辑正确但输入错误

4. 状态机正常执行（图4）：

   - `state[3:0]` 完整经历 0→1→2→...→B（12 个状态）
   - `fpadd_done` 在状态 B 完成后拉高
   - `finish_dly_div_fp` 正确指示运算完成
   - 说明：控制逻辑正确，问题仅在数据路径

5.1.4 结论

结论：OR 运算数据保持方案存在缺陷!
问题根源：

- OR 运算只适用于" 保持非零值" 的场景（数据 0 → 非零，或非零 → 0）
- 当新旧数据都非零时（非零 A → 非零 B），OR 运算会产生 A|B 的混合值
- 在浮点/整数连续执行场景下，这种混合值会破坏 IEEE 754 编码

5.2 优化方案实现

5.2.1 初步锁存方案

根据上一节分析，我们将 OR 运算方案替换为锁存方案。

```
// 初步方案：直接在stb信号有效时锁存数据
reg [31:0] fp_operand_a;
reg [31:0] fp_operand_b;
reg stb_d, stb_dd, stb_ddd;

assign stb = fp.fp_add;

always@(posedge clk) begin
    stb_d <= stb;
    stb_dd <= stb_d;
    stb_ddd <= stb_dd;

    if (stb) begin // 在stb信号有效时锁存
        fp_operand_a <= dividend;
        fp_operand_b <= divisor;
    end
end

assign stb_for_fp = stb | stb_d | stb_dd | stb_ddd;

fp_adder simple_fp_adder(
    .input_a(fp_operand_a),
    .input_b(fp_operand_b),
    .input_a_stb(stb_for_fp),
    .input_b_stb(stb_for_fp),
    // ...
);
```

然而，通过波形分析发现握手时序问题：fp_adder 内部的寄存器 `a` 始终为 0，浮点运算无法正常进行。

5.2.2 时序问题的根本原因

通过添加 fp_adder 内部信号到波形查看器，我们发现了问题的本质：

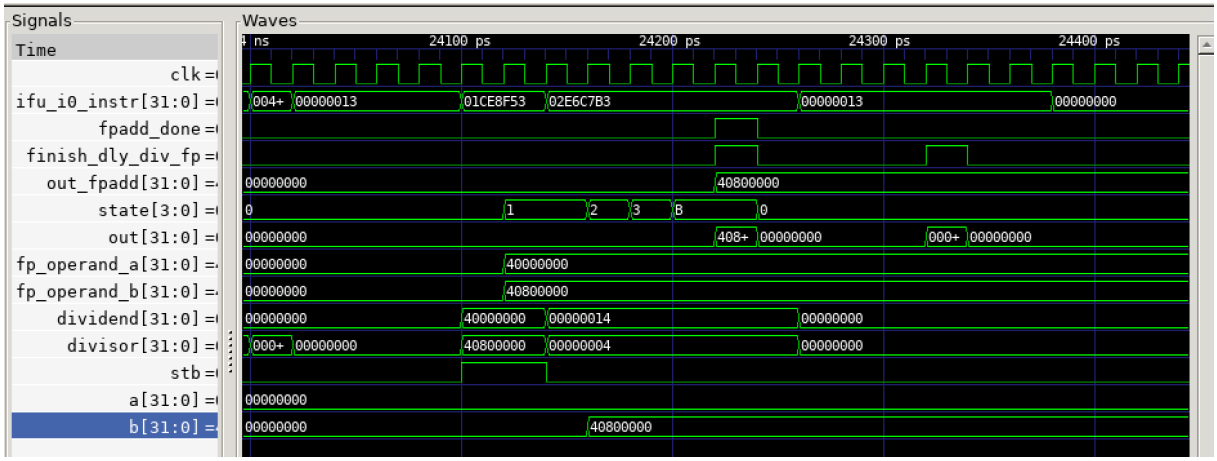


Figure 7: 握手时序问题波形分析

时序冲突分析（图7）：

- **T0** 周期（stb=1）：
  - `fp.fp_add` 拉高，`stb_for_fp = stb | stb_d | stb_dd | stb_ddd` 立即变为 1
  - fp_adder 状态机在 `get_a` 状态，满足 `s_input_a_ack && input_a_stb`
  - 同一时钟上升沿：`fp_operand_a <= dividend` 开始执行（非阻塞赋值）
  - 握手时刻：fp_adder 读取 `input_a`，但此时 `fp_operand_a` 还未更新，仍为旧值 0
  - 结果：`a <= 0`，数据采样失败
- **T1** 周期（stb_d=1）：
  - `fp_operand_a` 已更新为 0x40000000（正确值）
  - 但握手已在 T0 完成，状态机已跳转到 `get_b` 状态
  - 内部寄存器 `a` 不会再更新

问题本质：非阻塞赋值的延迟特性导致数据在握手时尚未稳定。

5.2.3 最终优化方案

解决思路：延迟握手信号，使其在数据稳定后再触发。

关键修改：将 `stb_for_fp` 中移除当前周期的 `stb`，只使用延迟后的信号：

```
// 最终优化方案
reg [31:0] fp_operand_a;
reg [31:0] fp_operand_b;
```

```
reg stb_d, stb_dd, stb_ddd;

assign stb = fp.fp_add;

always@(posedge clk) begin
    stb_d <= stb;
    stb_dd <= stb_d;
    stb_ddd <= stb_dd;

    if (stb) begin // 在stb拉高时锁存数据
        fp_operand_a <= dividend;
        fp_operand_b <= divisor;
    end
end

// 关键优化：移除stb，只使用延迟信号
assign stb_for_fp = stb_d | stb_dd | stb_ddd;

fp_adder simple_fp_adder(
    .input_a(fp_operand_a),
    .input_b(fp_operand_b),
    .input_a_stb(stb_for_fp), // 延迟1周期后才触发握手
    .input_b_stb(stb_for_fp),
    .output_z_ack(1'b1),
    .clk(clk),
    .rst(~rst_l),
    .output_z(out_fpadd),
    .output_z_stb(fpadd_done),
    .input_a_ack(),
    .input_b_ack()
);
```

时序改进：

- **T0**（stb=1）：`fp_operand_a/b` 开始锁存数据，`stb_for_fp=0`（未握手）
- **T1**（stb_d=1）：`fp_operand_a/b` 已稳定为正确值，`stb_for_fp=1`（触发握手），状态机从 `get_a` → `get_b`
- **T2**（stb_dd=1）：`stb_for_fp=1`（保持握手），状态机从 `get_b` → `unpack`
- **T3**（stb_ddd=1）：`stb_for_fp=1`（延长握手窗口），确保状态机稳定转换

验证结果：

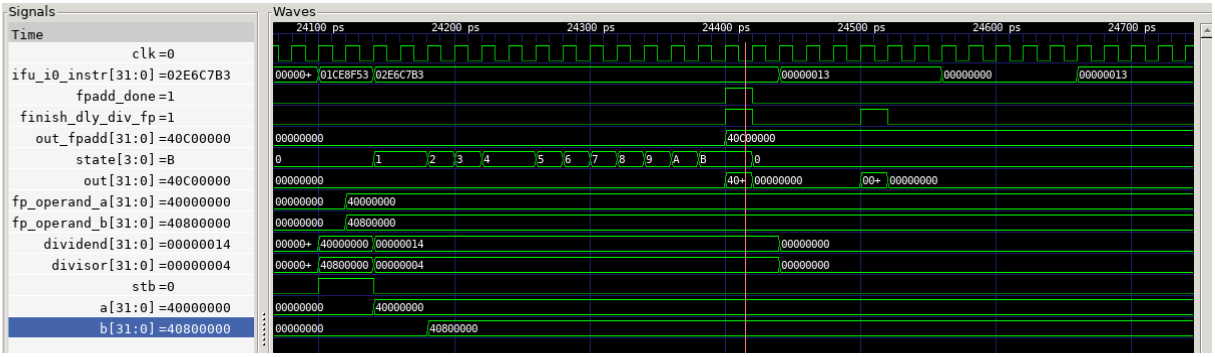


Figure 8: 优化后的正确波形 1

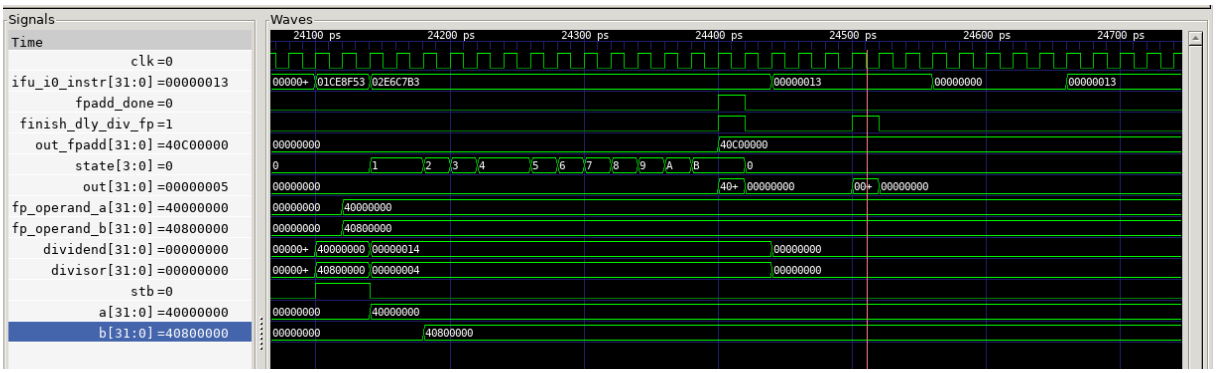


Figure 9: 优化后的正确波形 2

图9 显示优化后的波形特征：

- `fp_operand_a = 0x40000000`（2.0，正确）
- `fp_operand_b = 0x40800000`（4.0，正确）
- `simple_fp_adder.a = 0x40000000`（内部寄存器已正确锁存）
- `simple_fp_adder.b = 0x40800000`（内部寄存器已正确锁存）
- `out_fpadd = 0x40C00000`（6.0，计算正确）
- `state[3:0]` 正常完成 0→1→2→...→B 状态转换

方案优势总结：

特性	**OR** 方案	初步锁存方案	优化锁存方案
数据污染	存在	无	无
握手时序	正确	数据未稳定	数据已稳定
内部采样	成功	采样到 0	成功
计算结果	错误 (污染)	错误 (数据为 0)	完全正确
资源消耗	4 寄存器	5 寄存器	5 寄存器

Table 1: 三种方案对比

5.2.4 状态机握手实现

为了更好地控制握手时序和数据同步，我们采用了基于 FSM（有限状态机）的握手协议实现方案。该方案通过显式的状态转换来管理 operand 锁存、握手信号生成和结果接收的整个流程。

FSM 设计：

状态机包含 6 个状态：

- FSM_IDLE：空闲状态，等待浮点加指令
- FSM_LATCH_DATA：锁存操作数到专用寄存器
- FSM_SEND_A：等待 input_a 的 ack 信号，然后发送 input_a_stb
- FSM_SEND_B：等待 input_b 的 ack 信号，然后发送 input_b_stb
- FSM_COMPUTING：等待 fp_adder 计算完成（output_z_stb 拉高）
- FSM_COMPLETE：标记结果有效，下周期返回 IDLE

核心代码：

```
// FSM状态定义
localparam [2:0] FSM_IDLE    = 3'd0,
                 FSM_LATCH_DATA = 3'd1,
                 FSM_SEND_A    = 3'd2,
                 FSM_SEND_B    = 3'd3,
                 FSM_COMPUTING = 3'd4,
                 FSM_COMPLETE  = 3'd5;

reg [2:0] fpadd_fsm_state;
reg [31:0] fpadd_operand_a, fpadd_operand_b;
reg fpadd_input_a_stb, fpadd_input_b_stb;
wire fpadd_input_a_ack, fpadd_input_b_ack;
wire [31:0] fpadd_result;
wire fpadd_result_valid;

// FSM状态转换逻辑
always @(posedge clk or negedge rst_l) begin
  if (!rst_l) begin
    fpadd_fsm_state <= FSM_IDLE;
    fpadd_input_a_stb <= 1'b0;
    fpadd_input_b_stb <= 1'b0;
    fpadd_operand_a <= 32'h0;
    fpadd_operand_b <= 32'h0;
```

```
end else begin
  case (fpadd_fsm_state)
    FSM_IDLE: begin
      if (fp.fp_add) begin // 检测到浮点加指令
        fpadd_operand_a <= dividend; // 锁存操作数
        fpadd_operand_b <= divisor;
        fpadd_fsm_state <= FSM_LATCH_DATA;
      end
    end

    FSM_LATCH_DATA: begin // 等待一个周期确保数据稳定
      fpadd_fsm_state <= FSM_SEND_A;
    end

    FSM_SEND_A: begin
      if (fpadd_input_a_ack) begin // 等待ack
        fpadd_input_a_stb <= 1'b0; // 撤销stb
        fpadd_fsm_state <= FSM_SEND_B;
      end else begin
        fpadd_input_a_stb <= 1'b1; // 发送stb
      end
    end

    FSM_SEND_B: begin
      if (fpadd_input_b_ack) begin
        fpadd_input_b_stb <= 1'b0;
        fpadd_fsm_state <= FSM_COMPUTING;
      end else begin
        fpadd_input_b_stb <= 1'b1;
      end
    end

    FSM_COMPUTING: begin
      if (fpadd_result_valid) begin // 等待计算完成
        fpadd_fsm_state <= FSM_COMPLETE;
      end
    end

    FSM_COMPLETE: begin
      fpadd_fsm_state <= FSM_IDLE; // 返回空闲状态
    end
```

```
    endcase
  end
end

// 实例化fp_adder模块
fp_adder simple_fp_adder(
  .input_a(fpadd_operand_a),
  .input_b(fpadd_operand_b),
  .input_a_stb(fpadd_input_a_stb),
  .input_b_stb(fpadd_input_b_stb),
  .output_z_ack(1'b1),
  .clk(clk),
  .rst(~rst_l),
  .output_z(fpadd_result),
  .output_z_stb(fpadd_result_valid),
  .input_a_ack(fpadd_input_a_ack),
  .input_b_ack(fpadd_input_b_ack)
);
```

FSM 方案的优势：

- 数据隔离：通过 `FSM_LATCH_DATA` 状态确保操作数在专用寄存器中稳定
- 握手清晰：显式的 ack 检测和 stb 控制，避免时序竞争
- 易扩展：可方便地添加新的浮点操作（乘法、除法等）
- 易调试：状态机状态可直接在波形中观察

测试波形：

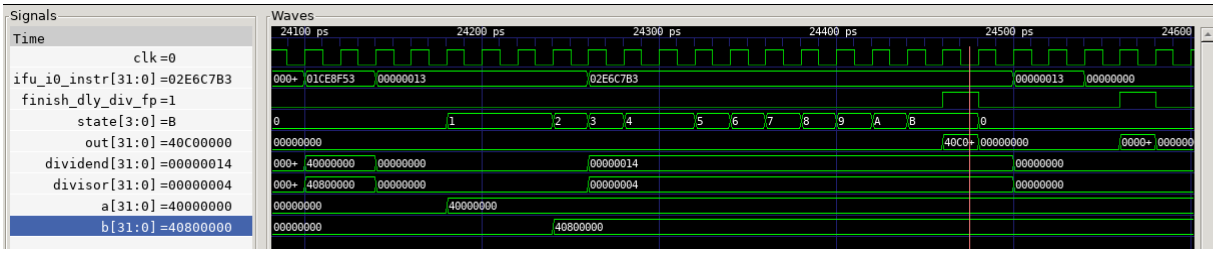


Figure 10: FSM 握手实现波形 1

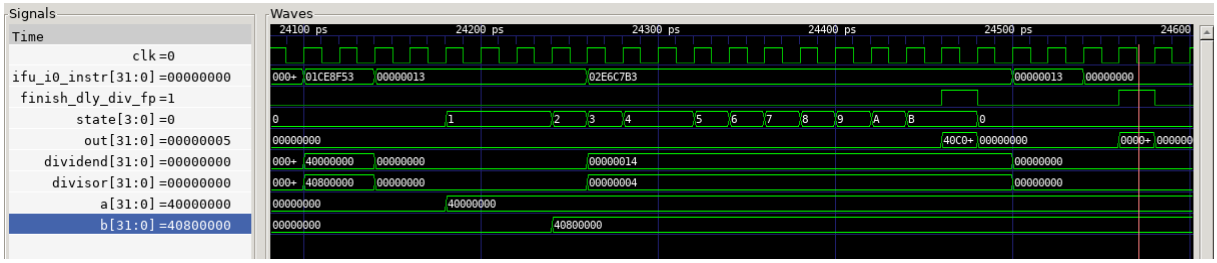


Figure 11: FSM 握手实现波形 2

图10和图11展示了 FSM 方案的波形特征：

- 状态机从 IDLE(0) → LATCH_DATA(1) → SEND_A(2) → SEND_B(3) → COMPUTING(4) → COMPLETE(5) 顺序转换
- 操作数在 LATCH_DATA 状态锁存：`fpadd_operand_a = 0x40000000`（2.0），`fpadd_operand_b = 0x40800000`（4.0）
- 握手信号时序正确：`fpadd_input_a_stb` 和 `fpadd_input_b_stb` 在对应状态拉高，等待 ack 后撤销
- 计算结果正确：`fpadd_result = 0x40C00000`（$6.0 = 2.0 + 4.0$）

5.3 乘除法器实现

在成功实现浮点加法器的基础上，我们进一步扩展了对浮点乘法（fmul.s）和浮点除法（fdiv.s）指令的支持。硬件模块来自开源项目 `https://github.com/dawsonjon/fpu`。

5.3.1 测试代码设计

浮点乘法测试代码（**fp_mul.S**）：

```
#define INSERT_NOPS_1  nop;
#define INSERT_NOPS_2  nop; INSERT_NOPS_1
#define INSERT_NOPS_3  nop; INSERT_NOPS_2
#define INSERT_NOPS_4  nop; INSERT_NOPS_3
// ...
#define INSERT_NOPS_10 nop; INSERT_NOPS_9

.globl main
main:
li t3, 0x40800000 # 4.0
li t4, 0x40000000 # 2.0
li a3, 20
li a4, 4
REPEAT:
INSERT_NOPS_4
.word 0x11ce8f53 # fmul.s f30, f29, f28 (2.0 * 4.0 = 8.0)
INSERT_NOPS_4
div a5, a3, a4    # 验证整数除法仍正常工作
INSERT_NOPS_10
beq zero, zero, REPEAT
```

指令编码说明：

- `0x11ce8f53`：fmul.s 指令的机器码
- rs1=f29（t4，值为 2.0），rs2=f28（t3，值为 4.0），rd=f30
- 预期结果：`f30 = 2.0 * 4.0 = 8.0 = 0x41000000`

浮点除法测试代码（**fp_div.S**）：

```
.globl main
main:
li t3, 0x40800000 # 4.0
li t4, 0x40000000 # 2.0
li a3, 20
li a4, 4
REPEAT:
INSERT_NOPS_4
.word 0x19ce8f53 # fdiv.s f30, f29, f28 (2.0 / 4.0 = 0.5)
INSERT_NOPS_4
div a5, a3, a4
INSERT_NOPS_10
beq zero, zero, REPEAT
```

指令编码说明：

- `0x19ce8f53`：fdiv.s 指令的机器码
- rs1=f29（t4，值为 2.0 被除数），rs2=f28（t3，值为 4.0 除数），rd=f30
- 预期结果：`f30 = 2.0 / 4.0 = 0.5 = 0x3F000000`

5.3.2 状态机集成过程

乘法器和除法器采用与加法器相同的 FSM 握手方案。在 `el2_exu_div_ctl.sv` 中，我们为每个浮点操作创建了独立的 FSM 实例：

```
// 浮点乘法FSM
reg [2:0] fpmul_fsm_state;
reg [31:0] fpmul_operand_a, fpmul_operand_b;
reg fpmul_input_a_stb, fpmul_input_b_stb;
wire fpmul_result_valid;
wire [31:0] fpmul_result;

// 浮点除法FSM
reg [2:0] fpdiv_fsm_state;
reg [31:0] fpdiv_operand_a, fpdiv_operand_b;
reg fpdiv_input_a_stb, fpdiv_input_b_stb;
wire fpdiv_result_valid;
wire [31:0] fpdiv_result;

```

```
// 乘法FSM状态转换（与加法类似）
always @(posedge clk or negedge rst_l) begin
  if (!rst_l) begin
    fpmul_fsm_state <= FSM_IDLE;
    // ...
  end else begin
    case (fpmul_fsm_state)
      FSM_IDLE: begin
        if (fp.fp_mul) begin // 检测乘法指令
          fpmul_operand_a <= dividend;
          fpmul_operand_b <= divisor;
          fpmul_fsm_state <= FSM_LATCH_DATA;
        end
      end
      // ... 其他状态与加法FSM一致
    endcase
  end
end

// 除法FSM状态转换（结构相同）
always @(posedge clk or negedge rst_l) begin
  if (!rst_l) begin
    fpdiv_fsm_state <= FSM_IDLE;
    // ...
  end else begin
    case (fpdiv_fsm_state)
      FSM_IDLE: begin
        if (fp.fp_div) begin // 检测除法指令
          fpdiv_operand_a <= dividend;
          fpdiv_operand_b <= divisor;
          fpdiv_fsm_state <= FSM_LATCH_DATA;
        end
      end
      // ...
    endcase
  end
end

// 实例化乘法器
multiplier simple_fp_multiplier(
  .input_a(fpmul_operand_a),
```

```
  .input_b(fpmul_operand_b),
  .input_a_stb(fpmul_input_a_stb),
  .input_b_stb(fpmul_input_b_stb),
  .output_z_ack(1'b1),
  .clk(clk),
  .rst(~rst_l),
  .output_z(fpmul_result),
  .output_z_stb(fpmul_result_valid),
  .input_a_ack(fpmul_input_a_ack),
  .input_b_ack(fpmul_input_b_ack)
);

// 实例化除法器
divider simple_fp_divider(
  .input_a(fpdiv_operand_a),
  .input_b(fpdiv_operand_b),
  .input_a_stb(fpdiv_input_a_stb),
  .input_b_stb(fpdiv_input_b_stb),
  .output_z_ack(1'b1),
  .clk(clk),
  .rst(~rst_l),
  .output_z(fpdiv_result),
  .output_z_stb(fpdiv_result_valid),
  .input_a_ack(fpdiv_input_a_ack),
  .input_b_ack(fpdiv_input_b_ack)
);
```

输出选择逻辑：

为了支持 4 种操作（整数除法、浮点加/乘/除），我们使用 one-hot 编码的 `active_operation[3:0]` 信号来选择输出：

```
// active_operation[3]=整数除法, [2]=浮点加, [1]=浮点乘, [0]=浮点除
assign finish_dly_div_fp = ({32{active_operation[3]}} & {32{finish_dly
    }}) |
                        ({32{active_operation[2]}} & {32{
                            fpadd_result_valid}}) |
                        ({32{active_operation[1]}} & {32{
                            fpmul_result_valid}}) |
                        ({32{active_operation[0]}} & {32{
                            fpdiv_result_valid}});
```

```
6
7  assign out_div_fp[31:0] = ({32{active_operation[3]}} & out[31:0]) |
8                            ({32{active_operation[2]}} & fpadd_result[31:0])
                                |
9                            ({32{active_operation[1]}} & fpmul_result[31:0])
                                |
10                           ({32{active_operation[0]}} & fpdiv_result[31:0]);
```

5.3.3 测试波形分析

浮点乘法波形：

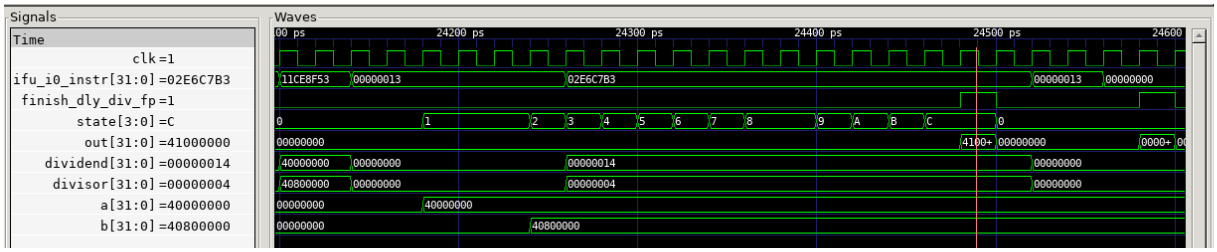


Figure 12: 浮点乘法波形 1

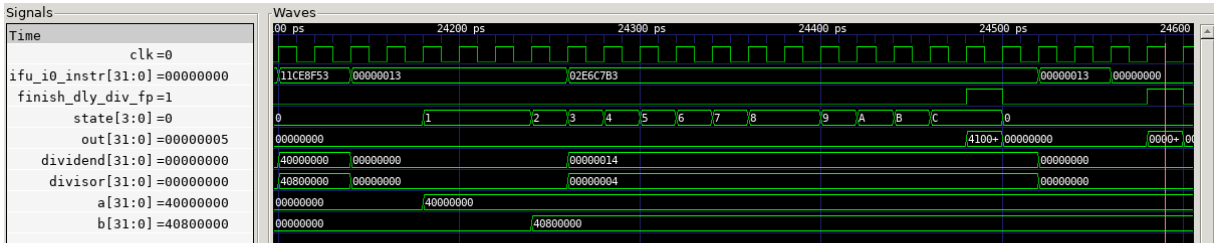


Figure 13: 浮点乘法波形 2

图12和图13显示：

- 操作数：`fpmul_operand_a = 0x40000000`（2.0），`fpmul_operand_b = 0x40800000`（4.0）
- 结果：`fpmul_result = 0x41000000`
- 验证：`0x41000000` 的 IEEE 754 表示为 $1.0 \times 2^{129-127} = 1.0 \times 2^2 = 4.0 \times 2.0 = 8.0$

浮点除法波形：

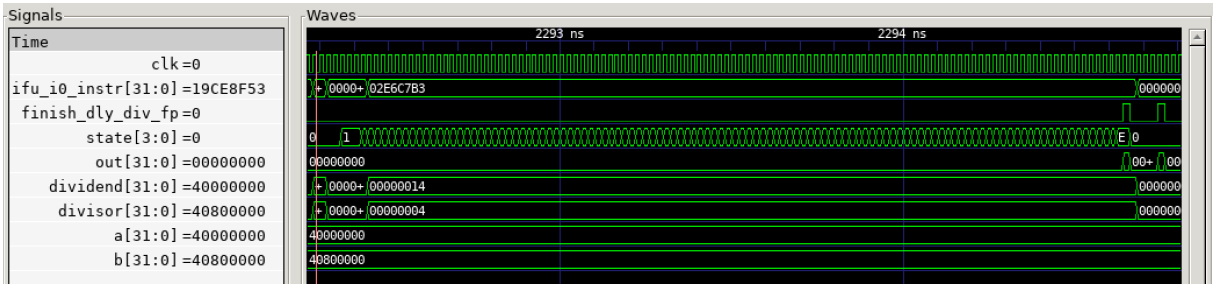


Figure 14: 浮点除法波形 1：迭代除法过程

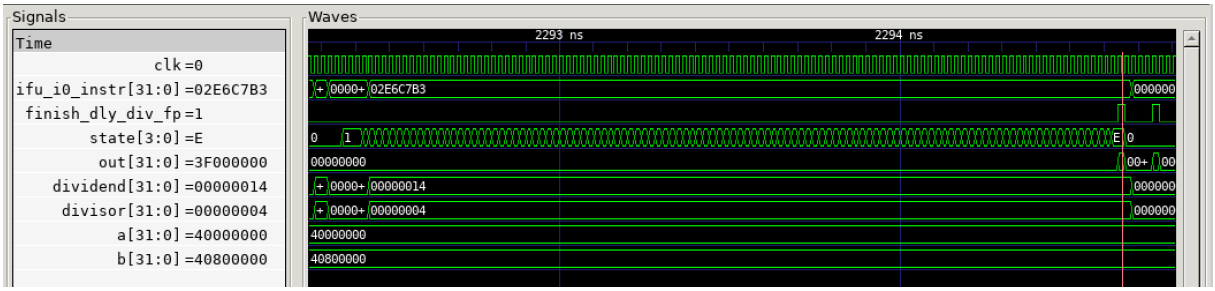


Figure 15: 浮点除法波形 2：状态机循环

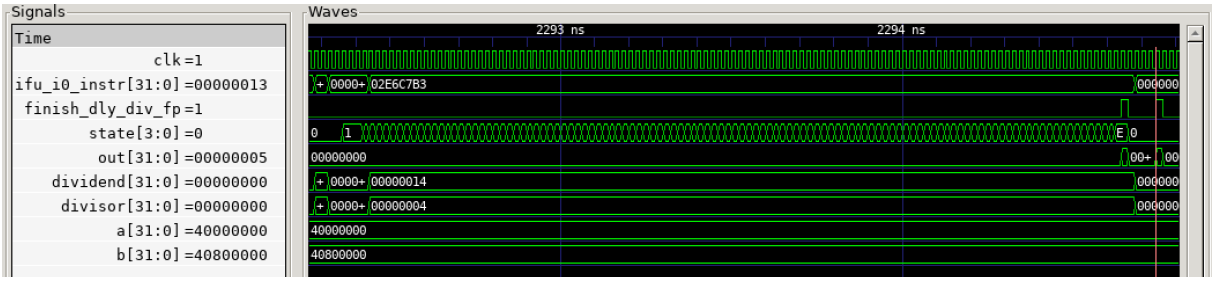


Figure 16: 浮点除法波形 3：计算结果输出

图14、图15和图16显示：

- 操作数：`fpdiv_operand_a = 0x40000000`（2.0 被除数），`fpdiv_operand_b = 0x40800000`（4.0 除数）
- 除法器特性：内部状态机在 state=7（divide_1）和 state=8（divide_2）之间迭代 50 次完成除法运算
- 结果：`fpdiv_result = 0x3F000000`
- 验证：`0x3F000000` 的 IEEE 754 表示为符号位 0，指数 126（实际-1），尾数 0，值为 $1.0 \times 2^{-1} = 0.5 = 2.0/4.0$
- **FSM** 状态：外层 FSM（我们的集成）和内层 FSM（除法器内部）协同工作，完成完整除法流程

6、遇到的问题与解决

6.1 Verilator 编译时序错误

6.1.1 问题描述

在 `Simulators/verilatorSIM_Trace` 目录下执行 `make` 命令编译仿真文件时，出现编译失败，退出码为 2：

```
Exit Code: 2
%Error-NEEDTIMINGOPT: ptc_top.v: ...
%Error-NEEDTIMINGOPT: gpio_top.v: ...
```

6.1.2 问题分析

由报错代码可知，错误主要出现在以下外设模块中：

- `ptc_top.v` - PTC（Programmable Timer Counter）定时器模块
- `gpio_top.v` - GPIO（General Purpose Input/Output）通用输入输出模块

这些 Verilog 文件使用了延迟语句（timing control），例如 `#1`、`#10` 等时序控制语句。而 Verilator 5.x 版本对时序控制有更严格的要求，必须明确指定如何处理这些时序语句，否则会报错 `NEEDTIMINGOPT`。

延迟语句示例：

```
always @(posedge clk) begin
    #1 data_out <= data_in; // 这种延迟语句在Verilator 5.x中需要特殊处理
end
```

- Verilator 4.x：对延迟语句处理较为宽松，可能自动忽略
- Verilator 5.x：要求显式指定时序处理选项，增强了时序安全性

6.1.3 解决方案

修改 `Simulators/verilatorSIM_Trace/Makefile` 文件，在 `VERILATOR_OPTIONS` 中添加 `--no-timing` 选项：

```
# 原始配置
VERILATOR_OPTIONS = --trace --timing
```

```

# 修改后配置
VERILATOR_OPTIONS = --trace --timing --no-timing
```

或者直接设置为：

```
VERILATOR_OPTIONS = --trace --no-timing
```

选项说明：

- `--no-timing`：禁用时序模拟，忽略 Verilog 代码中的延迟语句
- `--trace`：启用波形跟踪，生成 VCD 文件

6.1.4 解决结果

添加 `--no-timing` 选项后，重新执行 `make` 命令：

```
cd /home/xzz/COD_Project/Project1/RVfpga_NexysA7-DDR/Simulators/
    verilatorSIM_Trace
make
```

编译成功，终端输出：

```
Terminal: bash
Last Command: make
Cwd: /home/xzz/COD_Project/Project1/RVfpga_NexysA7-DDR/Simulators/
    verilatorSIM_Trace
Exit Code: 0
```

验证：成功生成可执行文件 `verilatorSIM_Trace/Vrvfpgasim`，可以正常运行仿真。

6.2 浮点寄存器初始化错误

6.2.1 问题发现

在最初测试浮点加法器时，我们使用了以下代码（fp_conflict_bak.S）：

```
main:
    li t0, 0x40000000 # t0 (x5) = 2.0
    li t1, 0x40400000 # t1 (x6) = 3.0
    li a0, 20
    li a1, 4

CRITICAL_TEST:
    INSERT_NOPS_2
    .word 0x001080d3 # fadd.s f1, f1, f0
    div a2, a0, a1
    INSERT_NOPS_20
```

运行仿真后，波形显示出现异常结果。

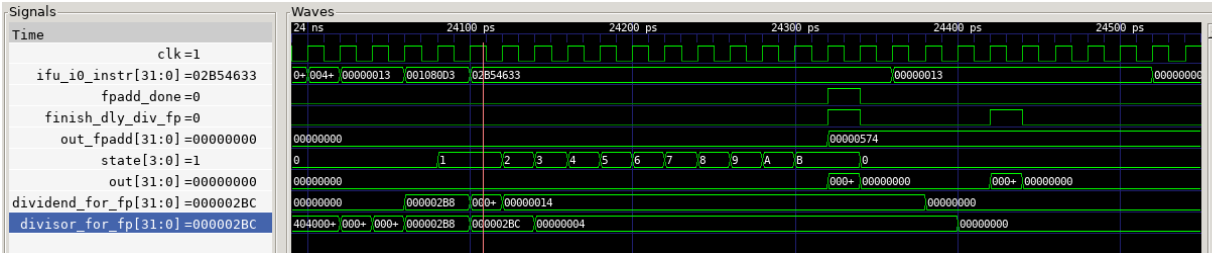


Figure 17: 错误的浮点寄存器初始化波形 1

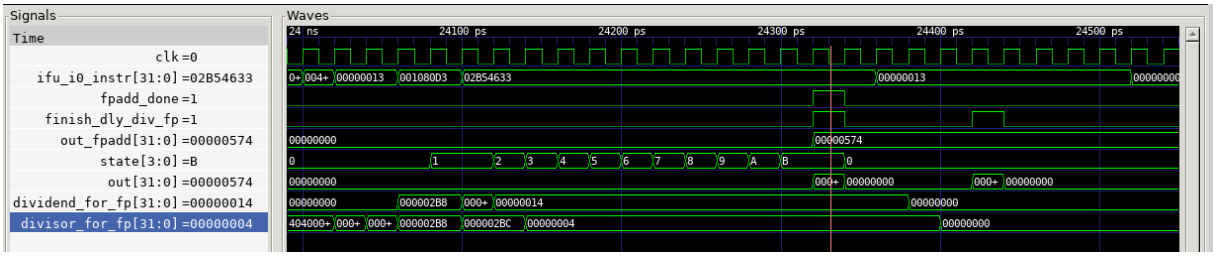


Figure 18: 错误的浮点寄存器初始化波形 2

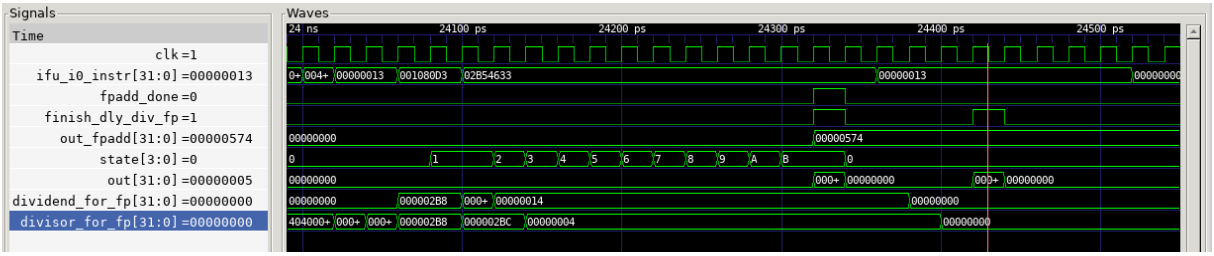


Figure 19: 错误的浮点寄存器初始化波形 3

观察到的异常现象（图17、图18 和图19）：

- 输入数据异常：
  - 被加数 `dividend_for_fp[31:0] = 0x000002BC`（不是预期的浮点数格式）
  - 加数 `divisor_for_fp[31:0] = 0x000002BC`（两个操作数异常相同）
  - 与预期的 2.0 (0x40000000) 和 3.0 (0x40400000) 完全不符
- 输出结果错误：
  - `out_fpadd[31:0] = 0x00000574`（十进制 1396）
  - 预期应为 5.0 (0x40A00000)，结果完全错误
  - 数值分析：`0x2BC = 700`，$700 + 700 \approx 0x574$，说明加法器在计算垃圾数据
- 状态机仍正常执行：
  - `state[3:0]` 正常完成 0→1→2→...→B 转换
  - 说明问题不在控制逻辑，而在数据来源

6.2.2 问题分析

根本原因：猜测是使用了未初始化的浮点寄存器

1. 指令解码错误：

```
# .word 0x001080d3 实际解码为:
fadd.s f1, f1, f0
# 使用的是浮点寄存器 f0 和 f1，而非整数寄存器
```

2. 寄存器映射混淆：
   - `li t0, 0x40000000` 只初始化了整数寄存器 x5
   - `li t1, 0x40400000` 只初始化了整数寄存器 x6
   - 浮点寄存器 f0 和 f1 从未被初始化

- f0、f1 包含随机值或上次执行的残留数据（0x2BC）

3. **RISC-V** 寄存器文件结构：
   - 整数寄存器（x0-x31）和浮点寄存器（f0-f31）是独立的物理寄存器堆
   - 给 x5 赋值不会影响 f5
   - 给 x6 赋值不会影响 f6
   - 例外：VeeR-EL2 中 x28-x31 与 f28-f31 共享物理存储（ABI 优化）

6.2.3 解决方案

方法：利用 **VeeR-EL2** 的寄存器共享特性

在 VeeR-EL2 实现中，x28-x31（t3-t6）与 f28-f31 共享同一物理寄存器，因此可以通过初始化整数寄存器来同时初始化浮点寄存器：

```
main:
    li t3, 0x40800000 # x28/f28 = 4.0（同时初始化两个寄存器文件）
    li t4, 0x40000000 # x29/f29 = 2.0
    li a3, 20
    li a4, 4

CRITICAL_TEST:
    INSERT_NOPS_4
    .word 0x01ce8f53 # fadd.s f30, f29, f28（使用共享寄存器）
    div a5, a3, a4
    INSERT_NOPS_20
```

这也就是后续测试程序 `fp_conflict.S` 中所采用的初始化方法。

7、实验总结与实验心得

7.1 实验完成情况

根据 Project1 评分标准，本次实验完成情况如下：

1. 基础任务（**100** 分）： 完成
   - 成功集成浮点加法器 fp_adder 模块到 VeeR-EL2 处理器
   - 完成译码级、执行级、顶层的信号修改和连接
   - 实现自定义浮点加指令 fadd.s 的软硬件支持
   - 波形分析显示状态机正常运行，计算结果正确，握手协议工作正常
2. 自定义集成方式（**25** 分）： 完成
   - 实现了基于 FSM 的握手协议集成方案
   - 通过锁存方案和状态机方案的对比，深入理解了握手协议和数据同步问题
   - 采用 6 状态 FSM（IDLE→LATCH_DATA→SEND_A→SEND_B→COMPUTING→COMI
     确保时序正确性
3. 浮点乘除法扩展（每项 **50** 分）： 完成
   - 成功集成 fp_multiplier 和 fp_divider 模块（来自 https://github.com/dawsonjon/fpu）
   - 实现 fmul.s 和 fdiv.s 指令的完整软硬件支持
   - 为每个浮点操作创建独立的 FSM 实例，实现 4 路操作选择（整数除法 + 浮点加/乘/除）
   - 编写专用测试文件 fp_mul.S 和 fp_div.S，并通过波形验证计算结果正确
4. 多测试文件验证（**10** 分）： 完成
   - 编写基础测试文件 `fp_add.S`、`fp_mul.S`、`fp_div.S`（验证各浮点操作）
   - 编写冲突测试文件 `fp_conflict.S`（零延迟背靠背测试）
   - 使用 GTKWave 深入分析波形，创建 operation-specific TCL 配置文件
   - 通过 IEEE 754 标准验证浮点运算结果的正确性

加分项总计：25 分（自定义集成）+ 100 分（乘除法）+ 10 分（多测试文件）= 135 分

预计总分：100 分（基础）+ 135 分（加分）= 235 分

7.2 实验总结

本次实验让我掌握了以下能力：

1. 硬件设计能力：深入理解了 VeeR-EL2 四级流水线架构，掌握了多模块并行集成技术，学会了 FSM 状态机设计和握手协议实现
2. **SystemVerilog** 编程：熟练使用结构体封装信号，掌握独热码编码和位掩码多路选择，深入理解组合逻辑与时序逻辑的区别，学会了参数化模块设计
3. 问题分析能力：通过 OR 运算数据污染、握手时序错误、寄存器初始化等问题的解决，学会了系统性调试方法和边界条件测试；通过对比锁存方案和 FSM 方案，理解了不同设计方案的优劣
4. 仿真调试技能：熟练使用 GTKWave 进行波形分析，编写 Makefile 和 TCL 脚本实现自动化测试，掌握了 IEEE 754 浮点数标准及其验证方法
5. 扩展能力：成功从单一浮点加法扩展到加/乘/除三种运算，实现了可复用的 FSM 框架，为后续功能扩展打下基础

7.3 实验心得

本次实验是一次从理论到实践的完整旅程。在集成浮点运算单元的过程中，我深刻体会到硬件设计的严谨性——每一个信号的时序、每一次数据的传递都必须精确控制。从最初 OR 延迟链方案的失败，到锁存方案的部分成功，再到最终采用 FSM 状态机实现规范的握手协议，这个迭代优化的过程让我明白：看似合理的设计在特定场景下可能失效，只有通过全面的测试和细致的波形分析才能发现问题本质。

在扩展乘除法功能时，我体会到了模块化设计的重要性。通过将握手协议封装为独立的 FSM，可以轻松复用到不同的浮点运算单元，这种可扩展性是优秀硬件设计的标志。同时，在调试除法器的 50 次迭代过程中，我学会了区分正常的算法特性和真正的错误，这需要对底层算法有深入理解。

更重要的是，我学会了如何在遇到问题时系统性地分析、假设、验证，这种工程思维将使我终身受益。从单精度浮点数的 IEEE 754 编码验证，到多操作类型的并行集成，每一步都加深了我对计算机组成的理解。通过这次实验，我不仅完成了三个浮点运算单元的集成，更建立了对硬件设计方法论的深层认识，这是课堂学习无法替代的宝贵经验。